

Career OS — Foundational Product Artefacts

1. Vision

Vision Statement

Build a human-governed Career Operating System that continuously assists candidates across the full job-seeking lifecycle — from opportunity discovery to offer conversion — through persistent memory, event-driven workflows, and specialized cognitive workers.

The system acts as:

- an operational career companion,
- a persistent career intelligence layer,
- and a workflow orchestration platform for job seeking.

The candidate remains the decision-maker at all times.

The system drafts, analyzes, recommends, prepares, and tracks.

The human chooses.

Long-Term Direction

The long-term evolution of the platform is not merely:

- resume generation,
- cover letter automation,
- or application tracking.

The long-term evolution is:

- adaptive career intelligence,
- longitudinal candidate understanding,
- strategic market positioning,
- recruiter interaction optimization,
- and personalized career cognition.

The system progressively learns:

- what opportunities convert,

- what narratives resonate,
- what skills are under-positioned,
- which recruiters respond,
- and how the candidate performs over time.

Memory is the moat.

2. Product Philosophy

2.1 Human-Centered Governance

The candidate is the protagonist.

AI workers are supporting systems that reduce operational burden.

The platform must never feel:

- autonomous,
 - uncontrollable,
 - or opaque.
-

2.2 Drafts, Not Decisions

All outbound artifacts:

- resumes,
- cover letters,
- recruiter responses,
- LinkedIn changes,
- interview prep documents

must be:

- reviewable,
- editable,
- explainable,
- and approval-gated.

No autonomous outbound actions occur without human approval.

2.3 Shared Context Over Fragmented Tools

All workers operate against a shared candidate memory layer.

The platform avoids:

- repetitive prompting,
- disconnected workflows,
- duplicated profile management,
- and isolated tooling.

Workers compose through shared memory.

2.4 Events Drive Cognition

The platform is not a linear pipeline.

Workers activate based on:

- triggers,
- events,
- state transitions,
- and contextual signals.

Examples:

- New job discovered
 - Recruiter replied
 - Interview scheduled
 - Resume approved
 - Application submitted
-

2.5 Longitudinal Intelligence

The product improves over time by learning:

- recruiter response patterns,
- candidate conversion patterns,
- interview performance trends,
- narrative effectiveness,
- skill gap correlations,
- and compensation trajectories.

3. Core Product Requirements

3.1 Persistent Candidate Layer

The system must maintain:

- Master CV
 - Role preferences
 - Career goals
 - Skills
 - Projects
 - Employment history
 - Compensation expectations
 - Geographic preferences
 - Work authorization details
 - Historical applications
 - Historical artifacts
 - Interview history
-

3.2 Application Lifecycle Support

The system must support:

- opportunity discovery,
 - evaluation,
 - tailoring,
 - application submission,
 - recruiter interactions,
 - interview preparation,
 - offer tracking,
 - and longitudinal analytics.
-

3.3 Event-Driven Orchestration

The system must support:

- asynchronous worker activation,
 - event subscriptions,
 - lifecycle transitions,
 - runtime triggers,
 - approval interruptions,
 - and digest generation.
-

3.4 Human Approval Governance

The system must:

- prevent autonomous outbound actions,
 - provide review/edit cycles,
 - support artifact regeneration,
 - provide provenance visibility,
 - and maintain approval history.
-

3.5 Shared Memory Access

Workers must operate against:

- shared candidate context,
 - shared application context,
 - shared artifact history,
 - and shared interaction history.
-

3.6 Longitudinal Learning

The system should accumulate intelligence around:

- candidate preferences,
- recruiter behavior,
- conversion rates,
- narrative success patterns,
- interview outcomes,
- and market positioning.

4. Capability Model

4.1 Discovery Capabilities

Workers

- Job Alert Aggregator
- LinkedIn Optimizer

Responsibilities

- opportunity ingestion,
 - role ranking,
 - keyword monitoring,
 - recruiter visibility tracking,
 - and market signal aggregation.
-

4.2 Evaluation Capabilities

Worker

- Resume Match Scorer

Responsibilities

- CV vs JD evaluation,
 - fit analysis,
 - gap detection,
 - and qualification scoring.
-

4.3 Transformation Capabilities

Workers

- Tailored Resume Builder
- Cover Letter Generator

Responsibilities

- role-specific artifact generation,
 - narrative adaptation,
 - and positioning optimization.
-

4.4 Preparation Capabilities

Workers

- Interview Prep Assistant
- Company Research Assistant

Responsibilities

- interview simulation,
 - STAR preparation,
 - company intelligence,
 - and recruiter preparation.
-

4.5 Tracking Capabilities

Worker

- Application Tracker

Responsibilities

- lifecycle tracking,
 - reminders,
 - status management,
 - follow-up orchestration,
 - and digest generation.
-

5. User Mental Model

The platform should feel like:

- an operational assistant,
- a career workspace,

- and a persistent support system.

It should NOT feel like:

- autonomous AI,
 - agent swarms,
 - or opaque automation.
-

UX Expectations

The user should always understand:

- why something surfaced,
 - what generated it,
 - what context was used,
 - and what actions remain under their control.
-

Cognitive Load Principles

The platform should:

- reduce repetition,
- reduce fragmentation,
- reduce organizational overhead,
- and reduce operational fatigue.

The platform should not:

- overwhelm the user with AI complexity,
 - expose orchestration internals,
 - or require workflow engineering by the user.
-

6. Trust Operating Model

6.1 Approval Gate Enforcement

All outbound actions require:

- human review,

- human approval,
 - and optional editing.
-

6.2 Explainability

Generated artifacts should expose:

- source context,
 - rationale,
 - missing information,
 - and confidence signals where appropriate.
-

6.3 Reversibility

The system must support:

- regeneration,
 - version rollback,
 - artifact comparison,
 - and revision history.
-

6.4 Provenance

Artifacts should maintain:

- generation timestamps,
 - source references,
 - and modification history.
-

7. Cognitive Access Governance

7.1 Scoped Memory Access

Workers access only the memory scopes relevant to their task.

Examples:

- Interview prep accesses interview/application context.
 - Resume tailoring accesses candidate profile + JD.
 - Job alerts access role preferences.
-

7.2 Contextual Retrieval

Retrieval should prioritize:

- active applications,
 - current candidate goals,
 - recent recruiter interactions,
 - and relevant artifacts.
-

7.3 Artifact Visibility Rules

Artifacts should maintain:

- ownership boundaries,
 - version lineage,
 - and contextual relationships.
-

8. Interaction Model

8.1 Event-Driven Interactions

The system operates through:

- event generation,
 - worker activation,
 - review interruptions,
 - and digest surfacing.
-

8.2 Runtime Categories

Always-On Workers

Continuous background systems.

Examples:

- Job Alert Aggregator
 - LinkedIn Optimizer
 - Application Tracker
-

Per-Application Workers

Application-scoped runtime cognition.

Examples:

- Match Scorer
 - Resume Builder
 - Cover Letter Generator
-

Per-Interview Workers

Interview-stage cognition.

Examples:

- Interview Prep
 - Company Research
-

8.3 Human Intervention Points

The human intervenes at:

- approval gates,
 - edit cycles,
 - opportunity selection,
 - submission decisions,
 - and negotiation decisions.
-

8.4 Notification Philosophy

Notifications should be:

- actionable,
 - digest-oriented,
 - minimally disruptive,
 - and context-rich.
-

9. Domain Model

Primary Entities

Candidate

Persistent user identity and career representation.

Opportunity

External role/job representation.

Application

Central operational object representing pursuit of an opportunity.

Artifact

Generated or uploaded content.

Examples:

- resumes,
 - cover letters,
 - prep notes,
 - recruiter replies.
-

Recruiter

External communication entity.

Interview

Interview-stage workflow entity.

Event

System trigger or lifecycle transition.

Insight

Derived intelligence from accumulated history.

10. Application Lifecycle State Machine

Lifecycle States

1. Discovered

Opportunity identified.

2. Shortlisted

User expresses interest.

3. Evaluated

Fit scoring completed.

4. Tailoring

Resume/cover letter generation active.

5. Approved

Artifacts approved by candidate.

6. Submitted

Application submitted externally.

7. Recruiter Engaged

Recruiter interaction detected.

8. Interviewing

Interview workflows active.

9. Negotiating

Offer/recruiter negotiation phase.

10. Rejected

Opportunity closed unsuccessfully.

11. Accepted

Offer accepted.

12. Archived

Lifecycle completed and retained historically.

11. Event Architecture

11.1 Purpose

The Event Architecture defines:

- how cognition is triggered,
- how workers coordinate,
- how workflows progress,
- how state transitions occur,
- and how the system avoids becoming a linear agent chain.

The platform is not a sequential workflow engine.

It is an event-driven operational system.

11.2 Architectural Philosophy

Events Are The Primary Orchestration Primitive

Workers do not directly invoke each other in rigid chains.

Instead:

- events are emitted,
 - interested workers subscribe,
 - state transitions occur,
 - human approvals interrupt flow,
 - and the platform evolves asynchronously.
-

State Drives Activation

Workers activate based on:

- lifecycle state,
 - contextual triggers,
 - user actions,
 - and external signals.
-

Human Interruptions Are First-Class Events

Approval points are explicit orchestration boundaries.

Examples:

- ResumeApproved
 - CoverLetterRejected
 - ApplicationSubmissionConfirmed
 - ArtifactEdited
-

11.3 Core Event Categories

Candidate Events

- CandidateCreated
- CandidateProfileUpdated
- SkillAdded
- TargetRoleUpdated

Opportunity Events

- JobDiscovered
- JobRanked
- JobShortlisted
- JobDismissed

Application Events

- ApplicationCreated
- ApplicationSubmitted
- FollowUpScheduled
- RejectionReceived
- OfferReceived

Artifact Events

- ResumeGenerated
- ResumeApproved
- CoverLetterGenerated
- ArtifactArchived

Interview Events

- RecruiterReplyDetected
- InterviewScheduled
- InterviewPrepGenerated

Governance Events

- ApprovalRequested
- ApprovalGranted
- ApprovalRejected

Insight Events

- ConversionPatternDetected
 - SkillGapTrendDetected
-

11.4 Central Operational Insight

The Application is the operational anchor.

Most meaningful events attach to:

- an Application,
- or eventually become associated with one.

The Application becomes:

- the workflow boundary,
 - memory boundary,
 - orchestration boundary,
 - artifact boundary,
 - and analytics boundary.
-

11.5 Worker Runtime Contracts

Workers receive:

- triggering event,
- scoped memory,
- active application context,
- candidate profile,
- relevant artifacts,
- and governance constraints.

Workers emit:

- artifacts,
 - recommendations,
 - notifications,
 - insights,
 - state transitions,
 - and follow-up events.
-

11.6 Human Approval As A System Primitive

Approval is not UI logic.

It is a core orchestration concept.

Examples:

- ResumeApprovalRequested
 - ResumeApproved
 - ResumeRejected
 - ApplicationSubmissionApproved
-

12. Product Roadmap

```
Vision
↓
Product Philosophy
↓
Core Product Requirements
↓
Capability Model
↓
User Mental Model
↓
Trust Operating Model
↓
Cognitive Access Governance
↓
Interaction Model
↓
Domain Model
↓
Application Lifecycle State Machine
```

↓
Event Architecture
↓
Memory Model
↓
Worker Runtime Model
↓
Artifact Lifecycle Model
↓
Human Approval Governance Model
↓
Information Architecture
↓
UX/System Flows
↓
Notification & Digest Model
↓
Dashboard Model
↓
Behavior Specifications
↓
Technical Architecture
↓
Runtime / Infrastructure Architecture
↓
Implementation Strategy
↓
Execution Roadmap

13. Current Position In Roadmap

The current product understanding is mature enough to strongly formalize:

- Vision
- Product Philosophy
- Core Product Requirements
- Capability Model
- User Mental Model
- Trust Operating Model
- Cognitive Access Governance
- Interaction Model
- Domain Model
- Application Lifecycle State Machine
- Event Architecture

The next highest-leverage artefacts are:

- Memory Model
 - Worker Runtime Model
 - Artifact Lifecycle Model
 - Human Approval Governance Model
 - Information Architecture
-

14. Product Artefact Maturity Assessment

This section evaluates:

- which artefacts are mature enough for a V1 definition,
 - which artefacts are partially known,
 - and which artefacts still require discovery.
-

14.1 Fully Mature Enough For V1 Formalization

The following artefacts already contain sufficient conceptual clarity to produce strong V1 definitions.

Vision

Status: Ready for formalization.

Product Philosophy

Status: Ready for formalization.

Core Product Requirements

Status: Ready for formalization.

Capability Model

Status: Ready for formalization.

User Mental Model

Status: Ready for formalization.

Trust Operating Model

Status: Ready for formalization.

Cognitive Access Governance

Status: Ready for formalization.

Interaction Model

Status: Ready for formalization.

Domain Model

Status: Ready for formalization.

Application Lifecycle State Machine

Status: Ready for formalization.

Event Architecture

Status: Ready for formalization.

14.2 Partially Known — Strong Enough For V1 Drafts

These artefacts are already inferable from the current product direction, but still contain unresolved implementation or UX decisions.

They are still valuable to draft now because they stabilize future thinking.

Memory Model

What We Already Know

- Persistent candidate memory is central.
- Application-scoped memory exists.
- Artifact history matters.
- Recruiter interactions matter.
- Longitudinal intelligence is a key moat.
- Workers require scoped retrieval.
- Shared memory is foundational.

What Is Still Unknown

- Vector vs relational retrieval boundaries.
- Memory aging/archival strategy.
- Memory summarization policies.

- Retrieval prioritization semantics.
- Cross-application learning boundaries.
- User-visible memory editing.

Recommendation

Proceed with a V1 conceptual Memory Model now.

Worker Runtime Model

What We Already Know

- Workers are event-driven.
- Workers are runtime-scoped.
- Workers operate through contracts.
- Workers require governance boundaries.
- Workers consume scoped context.
- Workers emit events and artifacts.

What Is Still Unknown

- Runtime framework choice.
- Queueing/orchestration implementation.
- Retry semantics.
- Parallel execution strategy.
- Worker arbitration rules.
- Cost budgeting logic.

Recommendation

Proceed with a V1 conceptual Worker Runtime Model now.

Artifact Lifecycle Model

What We Already Know

- Artifacts are versioned.
- Artifacts are editable.
- Artifacts require approval.
- Regeneration is supported.
- Provenance matters.

- Artifacts belong to applications.

What Is Still Unknown

- Diffing UX.
- Version retention policy.
- Collaborative editing semantics.
- Artifact archival rules.
- Export/import boundaries.

Recommendation

Proceed with a V1 conceptual Artifact Lifecycle Model now.

Human Approval Governance Model

What We Already Know

- No autonomous outbound actions.
- Approval gates are first-class.
- Human review interrupts workflows.
- Approval lineage matters.
- Trust is foundational.

What Is Still Unknown

- Confidence thresholds.
- Auto-suggestion aggressiveness.
- Multi-step approval flows.
- Delegated approval scenarios.
- Future automation relaxation policies.

Recommendation

Proceed with a V1 governance model now.

Information Architecture

What We Already Know

- Applications are the operational center.
- Persistent candidate workspace exists.

- Event-driven dashboards exist.
- Digests and approvals are key UX surfaces.
- Workers are background capabilities.
- The user should not see orchestration complexity.

What Is Still Unknown

- Primary navigation structure.
- Workspace hierarchy.
- Inbox vs dashboard philosophy.
- Mobile-first vs desktop-first priorities.
- Notification center semantics.
- Cross-application views.

Recommendation

Strong enough for a V1 IA draft.

14.3 Not Yet Mature Enough For Strong V1 Definitions

These artefacts still require substantial product or technical discovery.

Technical Architecture

Missing Inputs

- Expected scale.
- Concurrency assumptions.
- Worker execution volume.
- Budget constraints.
- Multi-tenant strategy.
- Compliance/security requirements.
- Hosting/runtime preferences.

Recommendation

Do not deeply formalize yet beyond high-level directional architecture.

Runtime / Infrastructure Architecture

Missing Inputs

- Event throughput expectations.
- Queueing strategy.
- Scheduling patterns.
- Blob/document storage volumes.
- Observability requirements.
- Retry/replay semantics.
- SLA expectations.

Recommendation

Too early for detailed infrastructure design.

Execution Roadmap

Missing Inputs

- Team size.
- Budget.
- Funding assumptions.
- Build timeline.
- Engineering velocity.
- GTM priorities.
- Technical staffing mix.

Recommendation

Can only be drafted at a very high level currently.

15. Memory Model — V1

15.1 Purpose

The Memory Model defines:

- what the system remembers,
- how memory is organized,
- how workers retrieve context,

- how longitudinal intelligence emerges,
- and how the platform evolves beyond stateless automation.

The Memory Model is one of the foundational architectural primitives of the Career OS.

The product moat is not generated resumes.

The moat is persistent career understanding.

15.2 Foundational Philosophy

Memory Is The Product

The system becomes increasingly valuable over time because it accumulates:

- candidate preferences,
- recruiter interactions,
- application history,
- successful narratives,
- interview outcomes,
- and behavioral patterns.

The system should progressively reduce operational friction while improving personalization and strategic intelligence.

Memory Must Be Scoped

Workers should not access unlimited context.

Memory retrieval must be:

- contextual,
- relevant,
- bounded,
- and purpose-driven.

This preserves:

- trust,
- explainability,
- relevance,

- and cognitive efficiency.
-

Memory Evolves Over Time

Not all memory remains equally relevant forever.

The system must support:

- active memory,
 - historical memory,
 - summarized memory,
 - archived memory,
 - and derived intelligence.
-

Memory Must Remain Human-Governed

The candidate owns their career representation.

The platform should avoid becoming:

- opaque,
- uncontrollable,
- or psychologically invasive.

Memory visibility and editability should remain part of the long-term UX philosophy.

15.3 Core Memory Categories

Candidate Memory

Persistent identity-level memory.

Examples

- Master CV
- Skills
- Career history
- Education
- Certifications

- Geographic preferences
- Compensation expectations
- Work authorization
- Preferred role families
- Preferred industries
- Career goals
- Leadership narratives
- Personal positioning themes

Characteristics

- Long-lived
 - Frequently reused
 - Shared across applications
 - Slowly evolving
-

Application Memory

Scoped operational memory attached to a specific application.

Examples

- Job description
- Fit score
- Tailored resume versions
- Cover letter versions
- Recruiter conversations
- Follow-up reminders
- Interview notes
- Status changes
- Offer details
- Rejection rationale

Characteristics

- Highly contextual
 - Operationally active
 - Lifecycle-driven
 - Event-connected
-

Artifact Memory

Memory attached to generated or uploaded artifacts.

Examples

- Resume variants
- Cover letter variants
- STAR answer libraries
- Interview prep documents
- LinkedIn summaries
- Thank-you email drafts

Characteristics

- Versioned
 - Editable
 - Regeneratable
 - Approval-governed
-

Interaction Memory

Memory derived from human-system interactions.

Examples

- Frequently rejected suggestions
- Preferred writing tone
- Editing patterns
- Ignored recommendations
- Preferred job categories
- Approval behaviors
- Application pacing patterns

Characteristics

- Behavioral
 - Adaptive
 - Personalization-oriented
 - Longitudinal
-

Insight Memory

Derived intelligence accumulated over time.

Examples

- High-performing narratives
- Strongest converting role families
- Interview weakness patterns
- Recruiter response trends
- Compensation trajectory insights
- Skill gap correlations
- Industry alignment patterns

Characteristics

- AI-derived
 - Pattern-oriented
 - Strategic
 - Evolves continuously
-

15.4 Memory Scopes

Global Candidate Scope

Accessible across the platform.

Contains:

- persistent candidate identity,
 - preferences,
 - and foundational career information.
-

Application Scope

Restricted to a single application lifecycle.

Contains:

- application artifacts,

- recruiter interactions,
- interview workflows,
- and operational state.

This becomes the primary operational memory boundary.

Worker Runtime Scope

Temporary contextual retrieval used by workers.

Workers receive:

- scoped context,
- relevant artifacts,
- recent interactions,
- active lifecycle state,
- and governance constraints.

Workers should avoid excessive context ingestion.

Historical Scope

Long-term archived memory retained for:

- analytics,
 - pattern detection,
 - historical review,
 - and strategic intelligence.
-

15.5 Memory Retrieval Philosophy

Retrieval Must Be Contextual

The system should prioritize memory based on:

- active application relevance,
- lifecycle state,
- recency,

- candidate goals,
- recruiter interaction proximity,
- and artifact relevance.

Retrieval should avoid:

- indiscriminate memory dumping,
 - irrelevant historical context,
 - and excessive prompt inflation.
-

Retrieval Is Goal-Oriented

Different workers require different retrieval semantics.

Examples

Resume Builder

Prioritizes:

- target role,
- candidate strengths,
- successful prior narratives,
- and relevant project experience.

Interview Prep

Prioritizes:

- company context,
- interview stage,
- recruiter interactions,
- and prior behavioral examples.

Job Alerts

Prioritizes:

- evolving candidate preferences,
 - accepted/rejected opportunities,
 - and target role trends.
-

Retrieval Should Prefer Active Context

The platform should heavily prioritize:

- active applications,
- current opportunities,
- recent interactions,
- and ongoing workflows.

Historical memory should degrade in retrieval priority over time unless strategically relevant.

15.6 Memory Lifecycle

Active Memory

Operationally relevant memory currently driving workflows.

Examples:

- active applications,
 - ongoing recruiter interactions,
 - scheduled interviews.
-

Dormant Memory

Older but still retrievable memory.

Examples:

- past applications,
 - prior interview loops,
 - historical artifacts.
-

Summarized Memory

Compressed representations of older interactions.

Examples:

- summarized recruiter history,
- recurring interview themes,

- historical narrative performance.
-

Archived Memory

Long-term retained memory primarily used for:

- analytics,
 - trend detection,
 - and longitudinal intelligence.
-

Derived Memory

AI-generated insights synthesized from historical patterns.

Examples:

- strongest positioning themes,
 - recurring recruiter interest areas,
 - skill-gap trends.
-

15.7 Memory Ownership & Governance

Candidate Ownership

The candidate remains the owner of:

- uploaded artifacts,
 - generated artifacts,
 - profile data,
 - and interaction history.
-

Human Editability

Candidates should eventually be able to:

- edit persistent profile information,
- remove outdated context,

- archive applications,
 - and control visible memory.
-

Memory Transparency

The platform should gradually evolve toward exposing:

- why recommendations occurred,
 - what context was used,
 - and which historical signals influenced outputs.
-

Inferred Intelligence Governance

AI-derived insights should remain:

- assistive,
- non-authoritative,
- and reviewable.

The platform should avoid presenting inferred traits as objective truth.

15.8 Longitudinal Intelligence Layer

The highest-value future capability of the system is longitudinal career intelligence.

Over time the system should identify:

- which narratives convert best,
- which companies engage most,
- which industries align strongest,
- where interview performance weakens,
- and which skills produce the highest opportunity expansion.

Examples:

- "Platform ownership narratives increase callback rates."
- "Healthcare companies show higher recruiter engagement than fintech."
- "Behavioral interviews frequently stall on leadership examples."
- "Director-level applications outperform senior engineer applications."

This transforms the system from:

- operational automation

into:

- strategic career intelligence.
-

15.9 Conceptual Memory Architecture

The platform likely requires multiple conceptual memory layers.

Structured Memory

Deterministic operational data.

Examples

- applications
- statuses
- companies
- recruiters
- events
- timelines
- reminders

Likely relational and query-oriented.

Semantic Memory

Unstructured contextual understanding.

Examples

- resumes
- cover letters
- recruiter conversations
- interview notes
- STAR stories
- preparation documents

Likely embedding-oriented and retrieval-based.

Derived Memory

AI-generated intelligence synthesized from patterns.

Examples

- positioning strengths
- recurring gaps
- narrative performance
- recruiter engagement patterns

Likely continuously evolving.

15.10 Architectural Implications

The Memory Model strongly suggests:

- hybrid memory architecture,
- scoped retrieval systems,
- application-centered memory boundaries,
- event-connected memory updates,
- artifact version lineage,
- and longitudinal analytics pipelines.

The product should avoid:

- flat vector dumping,
 - stateless prompting,
 - and globally injected context.
-

15.11 Most Important Insight

The Career OS is fundamentally:

- a persistent memory system,
- with event-driven cognitive workers operating against that memory.

The workers are replaceable.

The memory graph is not.

16. Worker Runtime Model — V1

16.1 Purpose

The Worker Runtime Model defines:

- what a worker is,
- how workers execute,
- how workers retrieve context,
- how workers emit outputs,
- how workers coordinate through events,
- and how workers remain composable instead of tightly coupled.

Workers are not the product.

Workers are runtime cognitive capabilities operating against the Career OS memory and event system.

16.2 Foundational Philosophy

Workers Are Specialized Cognitive Functions

Workers should not behave like autonomous digital employees.

Workers are:

- bounded,
- scoped,
- event-driven,
- and operationally constrained.

Each worker performs a specific cognitive responsibility.

Examples:

- evaluation,
- transformation,
- enrichment,
- preparation,
- tracking,
- or insight generation.

Workers Are Stateless Executors Over Stateful Memory

Workers themselves should remain operationally lightweight.

Persistent state belongs to:

- the candidate memory layer,
- application memory,
- artifact history,
- and the event system.

Workers consume scoped memory.

Workers do not become memory silos.

Workers Coordinate Through Events

Workers should not directly orchestrate each other.

Workers:

- subscribe to events,
- process scoped context,
- emit outputs,
- and generate follow-up events.

This preserves:

- composability,
 - scalability,
 - extensibility,
 - and explainability.
-

Human Governance Overrides Worker Autonomy

Workers never possess unrestricted outbound authority.

Human approval remains:

- mandatory,
- interruptible,
- and operationally central.

16.3 Worker Categories

Always-On Workers

Persistent background systems.

Examples

- Job Alert Aggregator
- LinkedIn Optimizer
- Application Tracker

Characteristics

- schedule-driven
 - event-reactive
 - low-interruption
 - digest-oriented
-

Per-Application Workers

Workers scoped to a specific application lifecycle.

Examples

- Resume Match Scorer
- Tailored Resume Builder
- Cover Letter Generator

Characteristics

- highly contextual
 - artifact-heavy
 - application-bound
 - approval-driven
-

Per-Interview Workers

Workers activated during recruiter/interview stages.

Examples

- Company Research Assistant
- Interview Prep Assistant

Characteristics

- recruiter-contextual
 - preparation-focused
 - time-sensitive
 - interaction-aware
-

Longitudinal Intelligence Workers

Future strategic intelligence systems.

Examples

- Narrative Performance Analyzer
- Career Strategy Advisor
- Compensation Trend Analyzer

Characteristics

- historical
 - pattern-oriented
 - analytics-driven
 - insight-generating
-

16.4 Worker Runtime Lifecycle

1. Event Trigger

A worker activates due to:

- system events,
- user actions,
- schedules,
- lifecycle transitions,
- or external signals.

2. Context Resolution

The orchestration layer resolves:

- application scope,
 - candidate scope,
 - artifact scope,
 - active lifecycle state,
 - governance constraints,
 - and retrieval priorities.
-

3. Scoped Retrieval

Workers retrieve:

- only relevant context,
- relevant artifacts,
- recent interactions,
- and operationally necessary memory.

Workers should avoid excessive prompt inflation.

4. Cognitive Execution

Workers:

- analyze,
 - transform,
 - evaluate,
 - summarize,
 - generate,
 - or enrich context.
-

5. Output Generation

Outputs may include:

- artifacts,

- recommendations,
 - notifications,
 - summaries,
 - insights,
 - or state transition requests.
-

6. Governance Interruption

If required, outputs enter:

- approval flows,
 - editing flows,
 - or regeneration flows.
-

7. Event Emission

Workers emit:

- follow-up events,
 - lifecycle transitions,
 - notifications,
 - or insight updates.
-

16.5 Worker Runtime Contracts

Inputs

Workers receive:

- triggering event,
 - scoped memory,
 - active application context,
 - relevant artifacts,
 - candidate profile,
 - and governance constraints.
-

Outputs

Workers may emit:

- artifacts,
 - evaluations,
 - recommendations,
 - notifications,
 - lifecycle transitions,
 - or derived insights.
-

Constraints

Workers must respect:

- memory boundaries,
 - application scope,
 - approval requirements,
 - confidence constraints,
 - and outbound governance.
-

16.6 Runtime Context Prioritization

Workers should prioritize:

- active applications,
- recent interactions,
- high-confidence signals,
- and operationally relevant memory.

Workers should deprioritize:

- stale applications,
 - irrelevant historical artifacts,
 - and inactive recruiter conversations.
-

16.7 Worker Isolation Principles

Workers should remain:

- independently deployable,
- independently replaceable,
- and operationally decoupled.

The system should avoid:

- hard-coded worker chains,
 - tightly coupled orchestration,
 - and worker-specific memory silos.
-

16.8 Retry & Failure Philosophy

Worker failures should:

- not corrupt application state,
- not lose memory lineage,
- and not bypass governance.

Failures should be:

- retryable,
 - observable,
 - and event-visible.
-

16.9 Cost & Runtime Philosophy

The platform should eventually support:

- runtime prioritization,
- deferred execution,
- selective retrieval,
- model routing,
- and cost-aware cognition.

Not all workers require the same intelligence depth.

16.10 Most Important Runtime Insight

Workers are not autonomous actors.

Workers are:

- bounded cognitive runtimes operating against persistent memory,
 - coordinated through events,
 - and governed through human approval.
-

17. Artifact Lifecycle Model — V1

17.1 Purpose

The Artifact Lifecycle Model defines:

- how artifacts are generated,
- edited,
- versioned,
- approved,
- regenerated,
- archived,
- and operationally linked to applications.

Artifacts are one of the primary user-visible outputs of the system.

17.2 Core Artifact Types

Examples include:

- tailored resumes,
 - cover letters,
 - STAR answer sets,
 - interview preparation packets,
 - recruiter responses,
 - thank-you emails,
 - LinkedIn summaries,
 - and negotiation drafts.
-

17.3 Artifact Philosophy

Artifacts are:

- draft-first,
- human-governed,
- editable,
- versioned,
- and application-contextual.

Artifacts should never be treated as final autonomous outputs.

17.4 Artifact Lifecycle

1. Generated

Initial artifact produced by a worker.

2. Reviewed

Candidate reviews generated output.

3. Edited

Candidate modifies content.

4. Approved

Candidate approves artifact for use.

5. Submitted / Used

Artifact used externally.

6. Archived

Artifact retained historically.

7. Regenerated

New variant produced while retaining lineage.

17.5 Versioning Principles

Artifacts should maintain:

- version lineage,

- timestamps,
- associated applications,
- generation context,
- and approval history.

The platform should support:

- comparisons,
 - rollback,
 - and iterative refinement.
-

17.6 Artifact Contextuality

Artifacts belong primarily to:

- an application,
- an interview process,
- or a recruiter interaction.

Artifacts should inherit:

- relevant job context,
 - candidate positioning,
 - and lifecycle state.
-

17.7 Regeneration Philosophy

Regeneration should:

- preserve lineage,
- maintain historical context,
- and avoid destructive overwrites.

The system should distinguish between:

- revision,
 - regeneration,
 - and manual editing.
-

17.8 Approval Governance

Artifacts should support:

- explicit approval states,
 - approval history,
 - rejection flows,
 - editing loops,
 - and regeneration requests.
-

17.9 Artifact Intelligence

Over time the platform should identify:

- strongest-performing resume variants,
- strongest recruiter engagement patterns,
- effective narrative styles,
- and successful positioning strategies.

Artifacts become inputs into longitudinal intelligence.

18. Human Approval Governance Model — V1

18.1 Purpose

The Human Approval Governance Model defines:

- how trust is preserved,
- how automation is bounded,
- how human review interrupts workflows,
- and how outbound actions remain governed.

This is one of the foundational trust primitives of the Career OS.

18.2 Foundational Governance Philosophy

The system assists.

The human decides.

Workers may:

- recommend,
- draft,
- summarize,
- and prepare.

Workers do not autonomously:

- apply,
 - communicate externally,
 - negotiate,
 - or alter external identity.
-

18.3 Approval Boundaries

Approval gates should exist before:

- application submission,
 - recruiter responses,
 - LinkedIn modifications,
 - interview commitments,
 - and externally visible communications.
-

18.4 Approval States

Artifacts and actions may exist in:

- draft,
 - pending review,
 - approved,
 - rejected,
 - regenerated,
 - or archived states.
-

18.5 Explainability Requirements

Approval surfaces should expose:

- what generated the output,
 - what context was used,
 - what assumptions were made,
 - and why recommendations occurred.
-

18.6 Reversibility

The system should support:

- rollback,
- revision,
- regeneration,
- and historical comparison.

Human decisions should remain reversible whenever operationally possible.

18.7 Confidence & Risk Philosophy

Higher-risk actions should require:

- stronger review visibility,
- clearer provenance,
- and more explicit approval.

Examples:

- recruiter communications,
 - compensation negotiations,
 - and public profile changes.
-

18.8 Governance Events

Examples include:

- ApprovalRequested
- ApprovalGranted
- ApprovalRejected
- RegenerationRequested
- ArtifactEdited

- OutboundActionBlocked
-

18.9 Long-Term Governance Evolution

The system may eventually support:

- configurable automation tolerance,
- selective auto-approval,
- delegated review,
- and confidence-based recommendations.

However:

Human governance remains foundational.

19. Information Architecture — V1

19.1 Purpose

The Information Architecture defines:

- how the workspace is organized,
 - how users navigate operational state,
 - how applications are surfaced,
 - and how complexity is hidden behind coherent UX structures.
-

19.2 Foundational IA Philosophy

The user should experience:

- operational clarity,
- contextual focus,
- and progressive disclosure.

The user should not experience:

- orchestration complexity,
- agent-centric UX,
- or fragmented workflows.

19.3 Primary Workspace Objects

The platform is primarily organized around:

- Candidate Workspace
 - Applications
 - Opportunities
 - Artifacts
 - Interviews
 - Notifications
 - Insights
-

19.4 Operational Center

The Application is the primary operational workspace.

Applications become:

- memory containers,
- workflow containers,
- artifact containers,
- and interaction containers.

Most user activity converges around active applications.

19.5 Primary Navigation Areas

Potential primary workspace areas:

Dashboard

Operational overview.

Opportunities

Discovered and shortlisted jobs.

Applications

Active application workflows.

Interviews

Interview-stage workflows.

Artifacts

Generated resumes, cover letters, prep docs.

Insights

Longitudinal career intelligence.

Profile

Persistent candidate memory and preferences.

19.6 Dashboard Philosophy

The dashboard should prioritize:

- actionable state,
- pending approvals,
- recruiter activity,
- upcoming interviews,
- and operational urgency.

The dashboard should avoid:

- excessive analytics,
 - noisy AI activity feeds,
 - and orchestration internals.
-

19.7 Notification Philosophy

Notifications should be:

- contextual,
- actionable,
- digest-oriented,
- and operationally meaningful.

The platform should minimize interruption fatigue.

19.8 Approval UX Philosophy

Approval flows should feel:

- lightweight,
- review-centric,
- editable,
- and confidence-building.

The UX should reinforce:

- user control,
 - transparency,
 - and reversibility.
-

19.9 Progressive Disclosure

The system should expose complexity gradually.

Beginner users should experience:

- simplicity,
- operational guidance,
- and low cognitive load.

Advanced users may later access:

- deeper controls,
 - memory visibility,
 - and workflow customization.
-

19.10 Most Important IA Insight

The platform should feel like:

- a coherent career workspace.

Not:

- a collection of AI tools.
-

20. UX/System Flows — V1

20.1 Purpose

The UX/System Flows define:

- how users operationally move through the platform,
- how workers activate during workflows,
- where approvals interrupt flows,
- and how event-driven orchestration becomes visible user experience.

The system should feel operationally coherent.

Not like disconnected AI tooling.

20.2 Candidate Onboarding Flow

Goal

Initialize persistent candidate memory.

Flow

1. Candidate creates workspace.
2. Candidate uploads Master CV.
3. Candidate defines:
 - target roles,
 - industries,
 - compensation expectations,
 - geographic preferences,
 - work authorization.
4. System initializes candidate memory.
5. Job discovery systems activate.

Key Principles

- minimal onboarding friction,
 - progressive enrichment over time,
 - avoid overwhelming setup complexity.
-

20.3 Job Discovery Flow

Goal

Surface relevant opportunities continuously.

Flow

1. Job ingestion systems discover opportunities.
2. Opportunities are ranked against candidate profile.
3. Daily digest generated.
4. Candidate:
 - shortlist,
 - dismiss,
 - or ignore opportunities.
5. Shortlisting emits application initialization events.

Key Principles

- low cognitive load,
 - digest-first UX,
 - relevance prioritization.
-

20.4 Application Flow

Goal

Operationalize candidate pursuit of a role.

Flow

1. Candidate selects opportunity.
2. Application object created.
3. Match scorer evaluates fit.
4. Resume generation activates.
5. Cover letter generation activates.
6. Candidate reviews artifacts.
7. Candidate edits/regenerates if needed.
8. Candidate approves artifacts.
9. Candidate submits externally.
10. Application state transitions to Submitted.

Key Principles

- applications are operational centers,
 - approval interruptions are explicit,
 - generated artifacts remain editable.
-

20.5 Recruiter Engagement Flow

Goal

Activate interview-stage cognition.

Flow

1. Recruiter response detected.
2. Recruiter engagement event emitted.
3. Interview preparation workers activate.
4. Company research generated.
5. Interview prep generated.
6. Follow-up reminders scheduled.
7. Candidate tracks interview progress.

Key Principles

- context-aware preparation,
 - minimal operational friction,
 - recruiter-stage prioritization.
-

20.6 Interview Workflow

Goal

Support candidate preparation and coordination.

Flow

1. Interview scheduled.
2. Interview workspace activated.
3. STAR stories surfaced.
4. Company research generated.
5. Candidate reviews preparation materials.

6. Notes and feedback captured.
7. Interview outcomes recorded.

Key Principles

- contextual preparation,
 - operational readiness,
 - longitudinal learning.
-

20.7 Offer Workflow

Goal

Support final-stage decision making.

Flow

1. Offer detected or entered.
2. Compensation details captured.
3. Negotiation preparation generated.
4. Candidate compares opportunities.
5. Candidate accepts or rejects.
6. Application lifecycle finalized.

Key Principles

- decision support,
 - comparative visibility,
 - non-autonomous negotiation support.
-

21. Dashboard Model — V1

21.1 Purpose

The Dashboard Model defines:

- what operational state is surfaced,
- how urgency is communicated,
- and how the platform becomes an operational cockpit for the candidate.

21.2 Dashboard Philosophy

The dashboard should prioritize:

- actionable state,
- pending approvals,
- recruiter activity,
- interview readiness,
- and operational urgency.

The dashboard should avoid:

- excessive analytics,
 - AI noise,
 - and orchestration internals.
-

21.3 Primary Dashboard Sections

Active Applications

Current operational opportunities.

Pending Approvals

Artifacts awaiting candidate review.

Recruiter Activity

Recent recruiter interactions.

Upcoming Interviews

Scheduled interviews and preparation readiness.

Opportunity Digest

Newly discovered opportunities.

Follow-Up Alerts

Applications requiring action.

Strategic Insights

Longitudinal intelligence and recommendations.

21.4 Dashboard Prioritization

The dashboard should prioritize:

1. operational urgency,
 2. recruiter responsiveness,
 3. pending approvals,
 4. upcoming deadlines,
 5. active interview loops.
-

22. Notification & Digest Model — V1

22.1 Purpose

The Notification & Digest Model defines:

- how the platform communicates urgency,
 - how interruptions are minimized,
 - and how operational awareness is maintained.
-

22.2 Notification Philosophy

Notifications should be:

- actionable,
- contextual,
- digest-oriented,
- and minimally disruptive.

The platform should avoid:

- constant interruptions,
 - excessive AI alerts,
 - and operational noise.
-

22.3 Notification Categories

High Urgency

Examples:

- recruiter response,
- interview scheduling,
- pending submission deadlines.

Medium Urgency

Examples:

- pending approvals,
- follow-up reminders,
- artifact review requests.

Low Urgency

Examples:

- LinkedIn suggestions,
- market insights,
- longitudinal recommendations.

22.4 Digest Philosophy

Digests should summarize:

- new opportunities,
- recruiter activity,
- follow-up tasks,
- interview timelines,
- and operational priorities.

Digest delivery should reduce notification fatigue.

23. Workspace Model — V1

23.1 Purpose

The Workspace Model defines:

- how operational context is organized,
 - how applications become working containers,
 - and how users navigate career workflows.
-

23.2 Foundational Workspace Principle

The Application is the primary operational workspace.

Applications contain:

- artifacts,
 - recruiter interactions,
 - interview history,
 - lifecycle state,
 - notes,
 - reminders,
 - and contextual memory.
-

23.3 Workspace Hierarchy

Candidate Workspace

Top-level persistent career environment.

Contains:

- candidate profile,
 - opportunities,
 - applications,
 - insights,
 - and historical memory.
-

Application Workspace

Operational execution container.

Contains:

- job description,
 - tailored artifacts,
 - recruiter activity,
 - interview preparation,
 - and lifecycle history.
-

Interview Workspace

Sub-context within applications.

Contains:

- interview schedules,
 - prep materials,
 - notes,
 - and feedback.
-

23.4 Cross-Application Views

The platform should support:

- active pipeline views,
 - recruiter engagement comparisons,
 - application health summaries,
 - and historical conversion visibility.
-

24. Review & Approval UX Model — V1

24.1 Purpose

Defines:

- how approval interactions behave,
 - how editing loops operate,
 - and how trust is reinforced operationally.
-

24.2 Approval UX Philosophy

Approval flows should feel:

- lightweight,
- confidence-building,
- editable,
- and transparent.

The system should reinforce:

- user control,
 - explainability,
 - and reversibility.
-

24.3 Core Approval Actions

Candidates should be able to:

- approve,
 - reject,
 - edit,
 - regenerate,
 - compare,
 - and archive outputs.
-

24.4 Regeneration Philosophy

Regeneration should:

- preserve lineage,
 - avoid destructive overwrite,
 - and maintain revision visibility.
-

24.5 Explainability Surfaces

Approval surfaces should expose:

- generation context,
- rationale,

- source memory,
 - and relevant assumptions.
-

25. Behavior Specifications — V1

25.1 Purpose

Behavior Specifications define:

- expected operational behavior,
- state transitions,
- worker activation semantics,
- and approval interruptions.

These specifications bridge:

- product semantics,
 - and engineering execution behavior.
-

25.2 Example Behavioral Rules

Recruiter Reply Behavior

When recruiter communication is detected:

1. recruiter engagement event emitted,
 2. application state updated,
 3. interview prep workers activated,
 4. candidate notified.
-

Resume Rejection Behavior

When candidate rejects generated resume:

1. artifact state updated,
2. regeneration permitted,
3. lineage preserved,
4. approval flow paused.

Stale Application Behavior

When application inactivity threshold exceeded:

1. follow-up reminder generated,
 2. dashboard urgency updated,
 3. digest inclusion prioritized.
-

Offer Received Behavior

When offer detected:

1. application state updated,
 2. negotiation workflows activated,
 3. comparative insights surfaced.
-

26. Engineering Handoff Readiness Summary

The product definition is now sufficiently mature for:

- engineering architecture alignment,
- frontend system planning,
- orchestration implementation planning,
- and coordinated product development.

The platform now has formalized:

- operational semantics,
- memory semantics,
- orchestration semantics,
- governance semantics,
- workflow semantics,
- and workspace semantics.

The system is now coherently defined as:

A persistent, event-driven, human-governed Career Operating System.

27. Engineering Handoff Artefact Package

The following artefacts now form the V1 product handoff package:

Foundational Product Layer

- Vision
 - Product Philosophy
 - Core Product Requirements
 - Capability Model
 - User Mental Model
 - Trust Operating Model
 - Cognitive Access Governance
 - Interaction Model
-

Operational System Layer

- Domain Model
 - Application Lifecycle State Machine
 - Event Architecture
 - Memory Model
 - Worker Runtime Model
 - Artifact Lifecycle Model
 - Human Approval Governance Model
-

Product Structure Layer

- Information Architecture
 - UX/System Flows
 - Dashboard Model
 - Notification & Digest Model
 - Workspace Model
 - Review & Approval UX Model
 - Behavior Specifications
-

28. Most Important Product Insights Discovered

The most important architectural and product discoveries made during formalization:

The Application Is The Operational Center

Applications act simultaneously as:

- workflow containers,
 - memory boundaries,
 - orchestration boundaries,
 - artifact boundaries,
 - interaction boundaries,
 - and analytics boundaries.
-

Events Are The Primary Orchestration Primitive

The system is fundamentally:

- event-driven,
- asynchronous,
- and stateful.

Workers coordinate through events.

Memory Is The Moat

The true long-term value is:

- persistent candidate understanding,
 - longitudinal intelligence,
 - and adaptive personalization.
-

Workers Are Runtime Capabilities

Workers are:

- bounded cognitive runtimes,
- operating over persistent memory,
- coordinated through events,
- and governed through human approval.

They are not autonomous agents.

Human Governance Is Foundational

The system assists.

The human decides.

Approval, editability, explainability, and reversibility remain foundational system principles.

29. Current Product Definition Maturity

The product definition has progressed beyond:

- feature ideation,
- AI tooling concepts,
- and workflow sketches.

The system now possesses:

- coherent operational semantics,
- coherent orchestration semantics,
- coherent memory semantics,
- coherent governance semantics,
- and coherent workspace semantics.

This provides a strong conceptual foundation for engineering execution while preserving product consistency across future implementation decisions.

30. Recommended Next Layer Beyond Product Artefacts

Once engineering begins implementation planning, the next downstream layers will likely include:

- technical architecture,
- runtime architecture,
- orchestration infrastructure,
- integration architecture,
- observability strategy,
- and execution sequencing.

Those documents should be engineering-owned while remaining aligned to the product semantics defined in this document.

31. Recommended Next Artefact Sequence

The highest-leverage next artefacts are:

1. Memory Model
2. Worker Runtime Model
3. Artifact Lifecycle Model
4. Human Approval Governance Model
5. Information Architecture
6. UX/System Flows
7. Notification & Digest Model
8. Dashboard Model

These artefacts together will complete the operational product semantics layer.

Once those are stable, downstream technical architecture becomes significantly easier and more coherent.

16. Current Product Understanding Summary

The product understanding has now evolved beyond:

- AI workers,
- resume tooling,
- and workflow automation.

The system is now better understood as:

A persistent, event-driven, human-governed Career Operating System built around longitudinal candidate intelligence.

The core architectural primitives are:

- persistent memory,
- application-centered orchestration,
- event-driven cognition,
- approval-governed automation,
- scoped worker runtimes,
- and longitudinal behavioral intelligence.

The most important operational insight discovered so far:

The Application is the central operational object of the system.

It acts simultaneously as:

- workflow container,
- memory boundary,
- orchestration boundary,
- artifact boundary,
- interaction boundary,
- and analytics boundary.